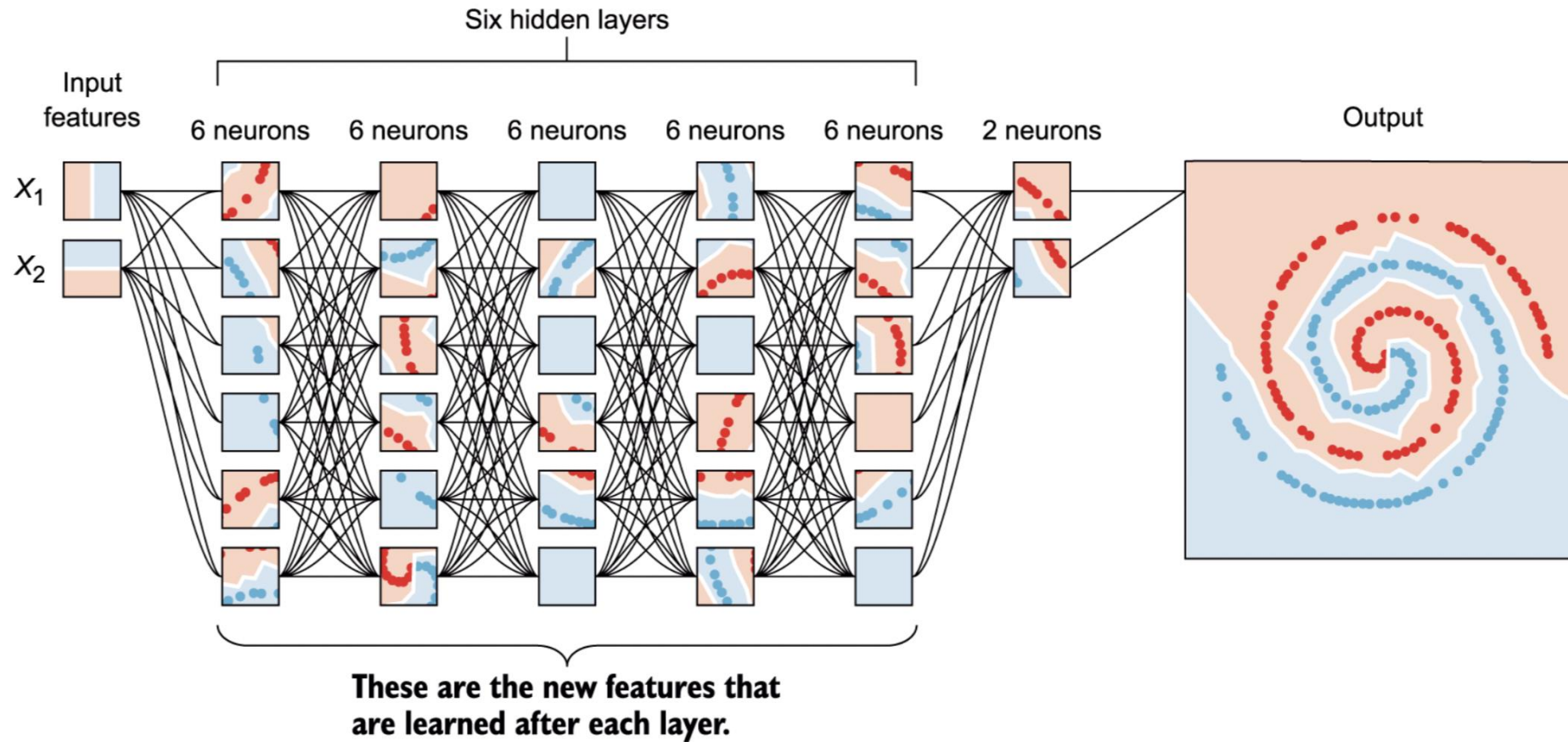


Deep Learning and Neural Networks

Priyanka Bagade, IITK

Lecture 4

Multilayer Perceptron Architecture



Input Layer

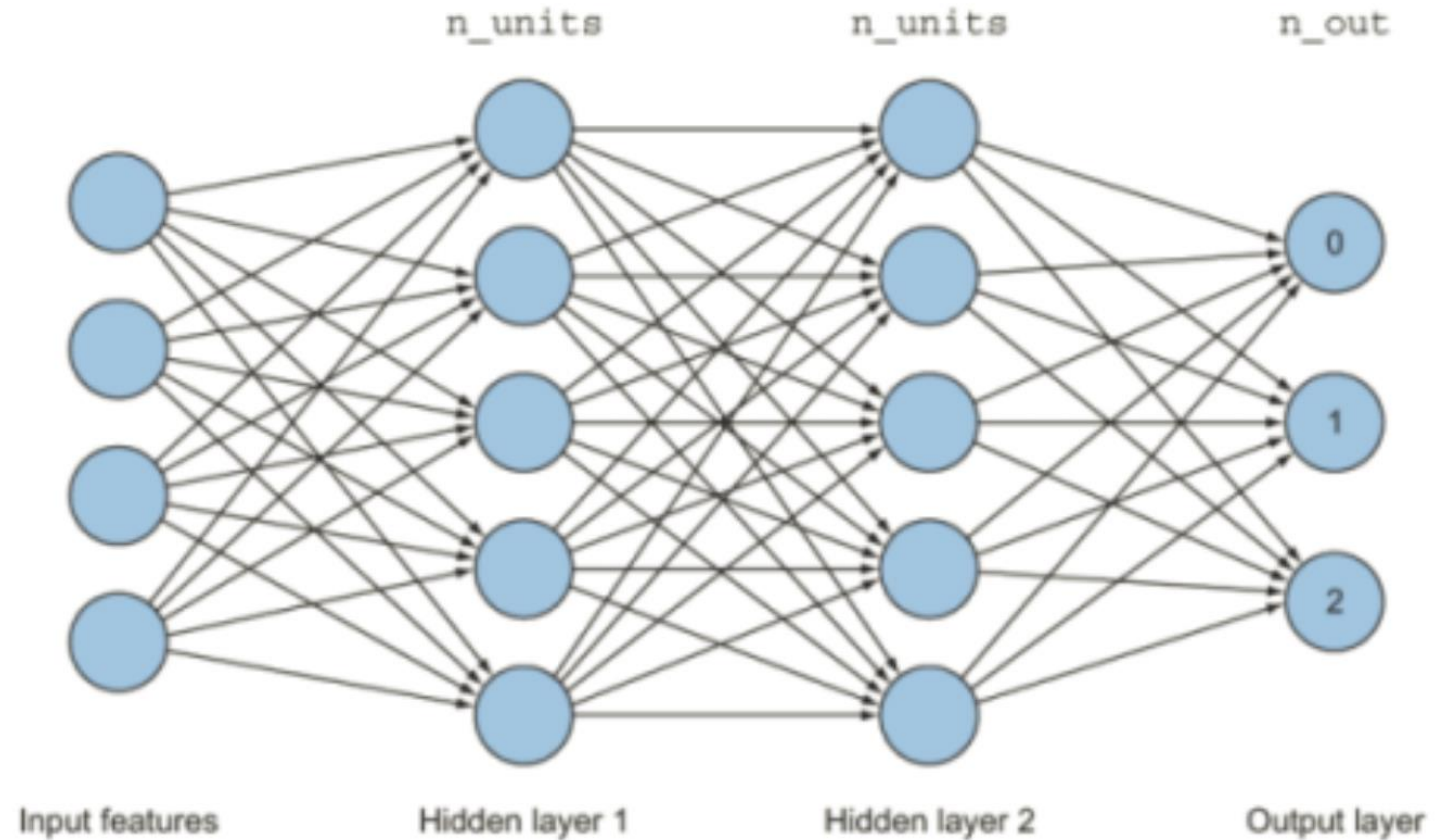
Weighted edges

Hidden Layers

Output Layer

Fully Connected Layers

- Total number of edges in the network
 - `Weights_0_1`:
(4 nodes in the input layer) $\times$ (5 nodes in layer 1)
+ 5 biases [1 bias per neuron]
= 25 edges
 - `Weights_1_2`:
5 $\times$ 5 nodes + 5 biases = 30 edges
 - `Weights_2_output`:
5 $\times$ 3 nodes + 3 bias = 18 edges
 - Total edges (weights) in this network = 73
- #weights = total number of trainable parameters in the network



Hyper Parameters

- Number of hidden layers
- Activation function
- Error Function
- Optimizer
- Batch size
- Number of epochs
- Learning rate

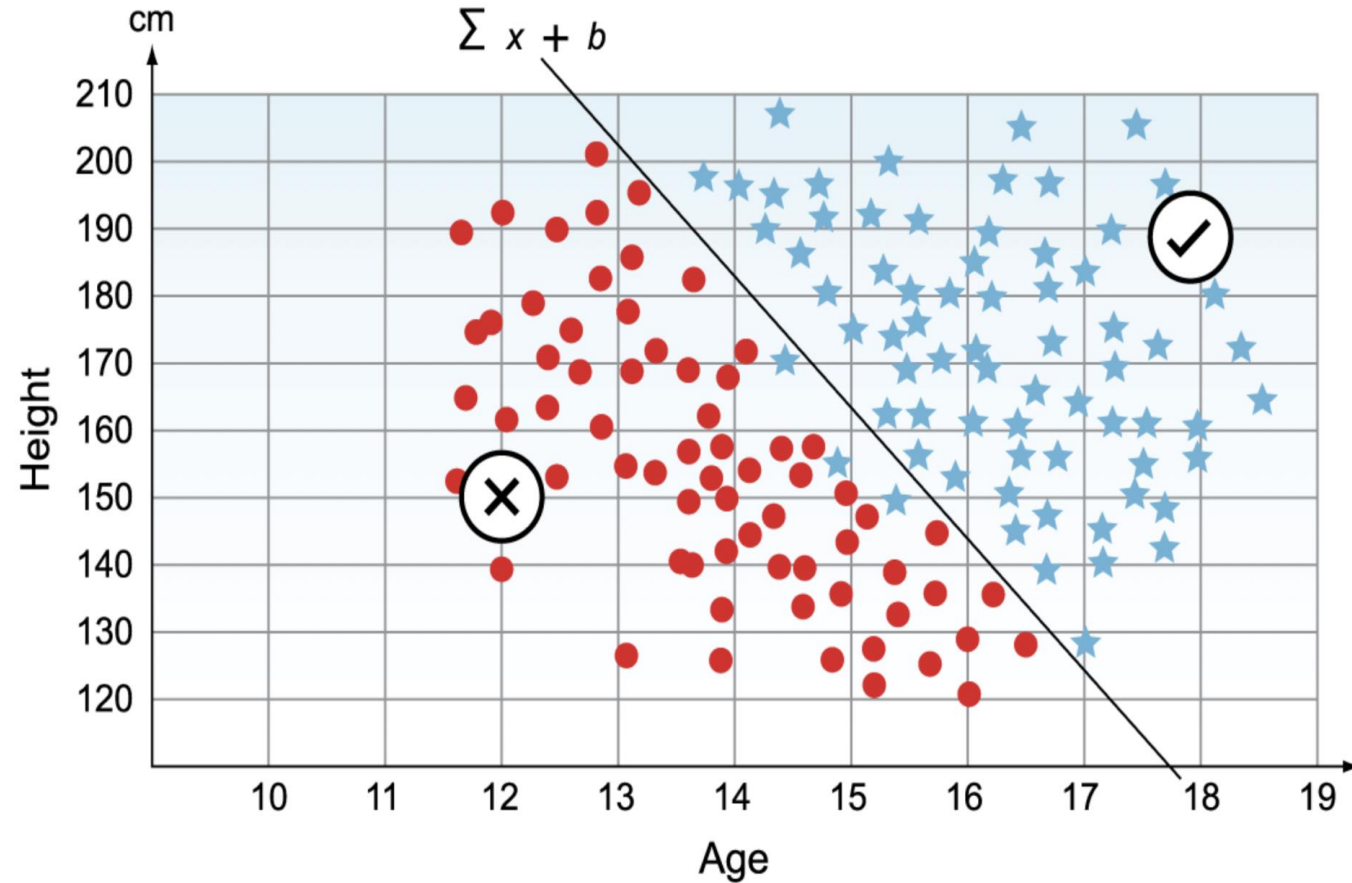


Activation Function

- Also called as transfer functions or nonlinearities
- Placed at the end of each perceptron to decide whether to activate it or not
- Without an activation function, the MLP will perform similar to a single perceptron
- Needed to restrict the o/p value to a certain finite value

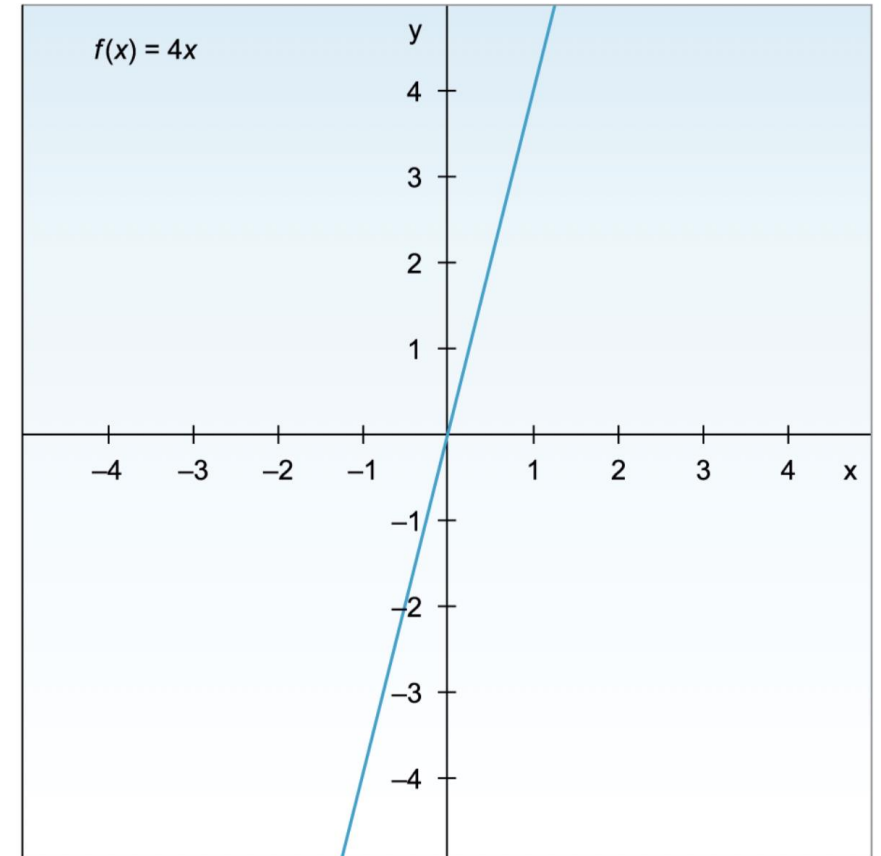
Activation Function

- Example: Player selection
 - $z = \text{height} * w1 + \text{age} * w2 + b$
 - z can take any value
 - If $z > 0$, fire the neuron
 - If $z < 0$, don't fire the neuron
 - But z alone is not taking any decision
 - Need an activation function to make the neuron firing decision

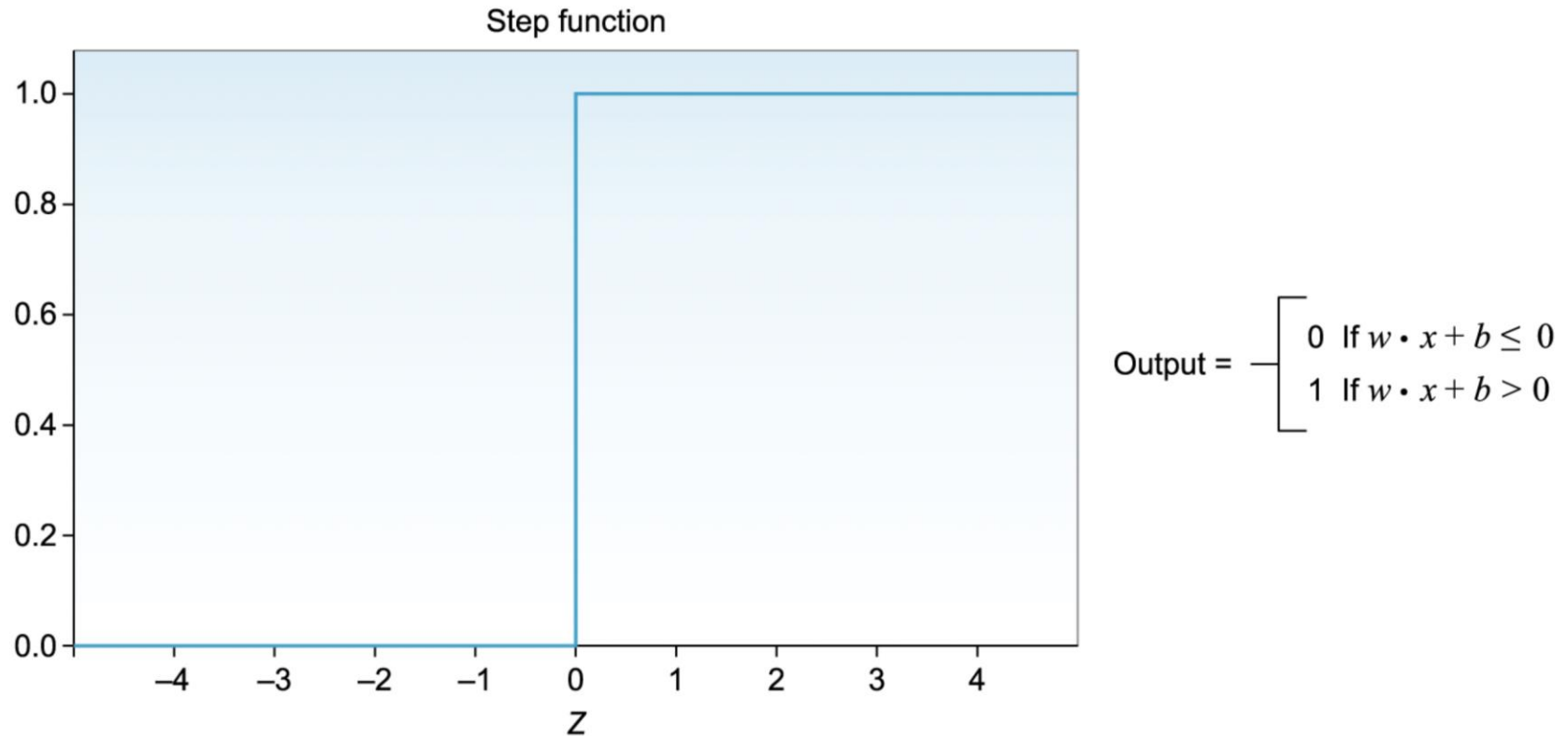


Linear Transfer function

- $\text{activation}(z) = z = w * x + b$
- If $w = 4$ and $b = 0$, the function plot will be...
- $z(x) = 4x$
- Derivative of $z(x)$ is $z'(x) = 4$, a constant
- Here the derivative does not depend on the input value x
- This leads to the issue of having same gradient descent
- That means we are not improving the error in the output



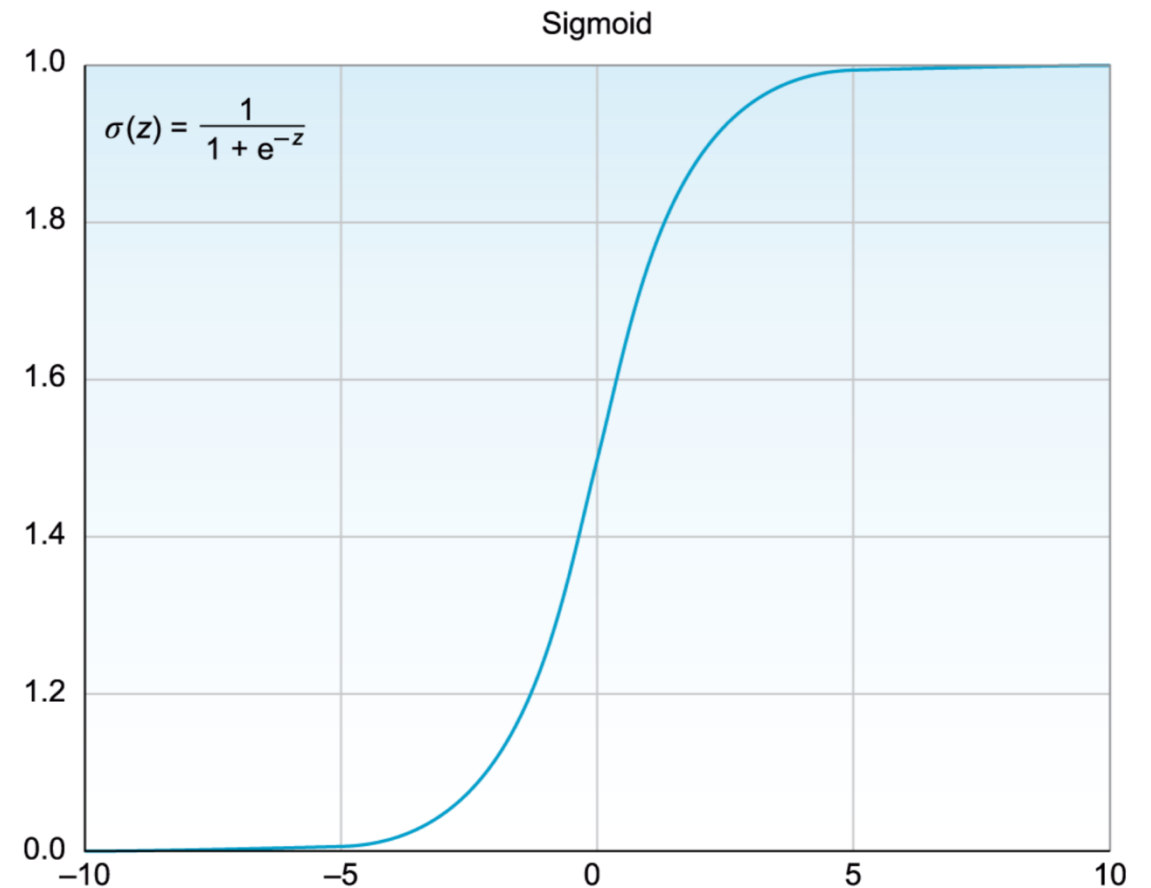
Step Activation Function (Binary classifier)



$\hat{y} = g(x)$, where g is an activation function and z is the weighted sum $= \sum x_i \cdot w_i + b$

Sigmoid/logistic function

- Step function – produces a discrete answer (pass or fail)
- Sigmoid function – produces the probabilities of passing and failing
- Provides the values of probabilities between 0 and 1

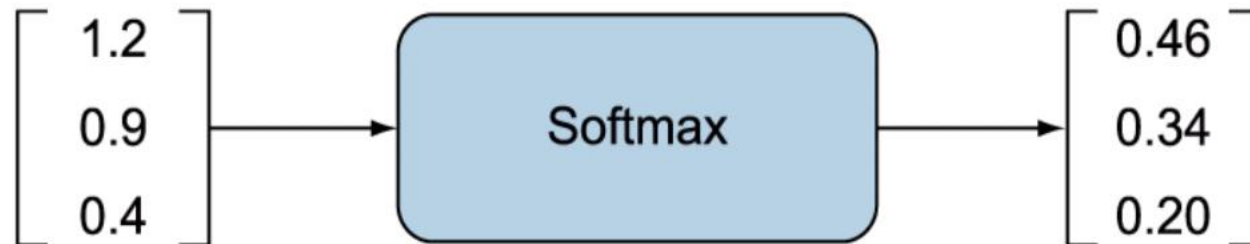


Softmax function

- A generalized sigmoid function
- Used when more than 2 classes in the classification problem
- Transforms the input values to probability values between 0 and 1
- o/p probabilities for all classes sum to 1
- Softmax function is given by

$$\sigma(x_j) = \frac{e^{x_j}}{\sum_i e^{x_i}}$$

- Normally used at the output layer

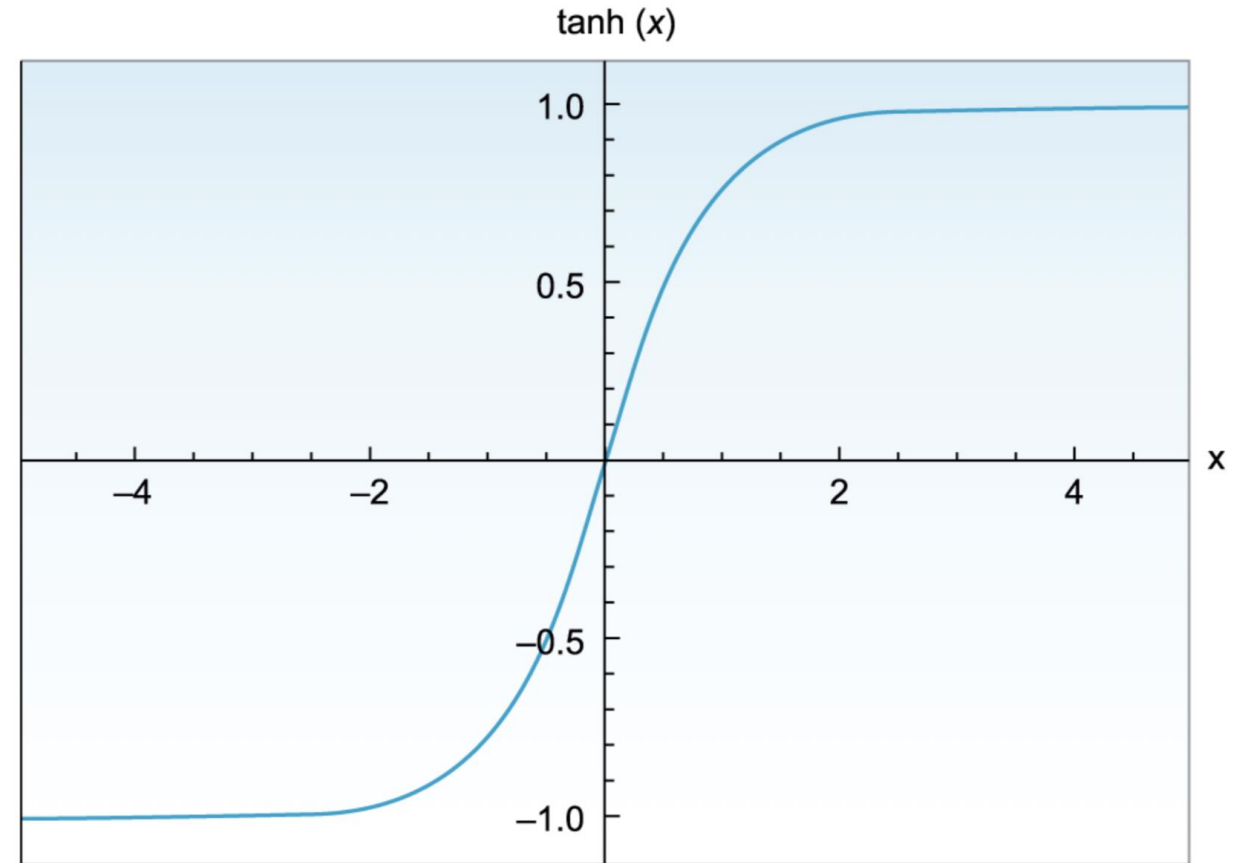


Hyperbolic tangent function(tanh)

- Shifted version of the sigmoid function
- Ranges the o/p values from -1 and 1
- Used for hidden layers

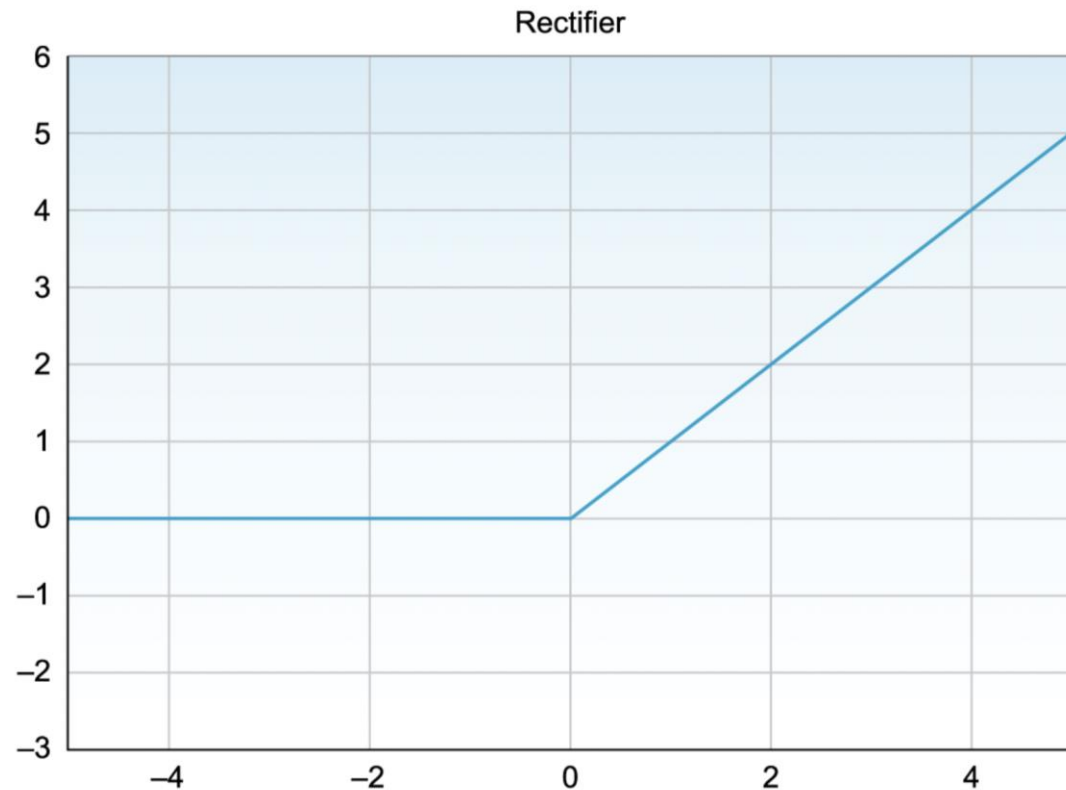
$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Drawback of sigmoid and tanh functions
 - If z is very large or small the derivative the function (gradient) becomes very small close to zero



Rectified linear unit (ReLU)

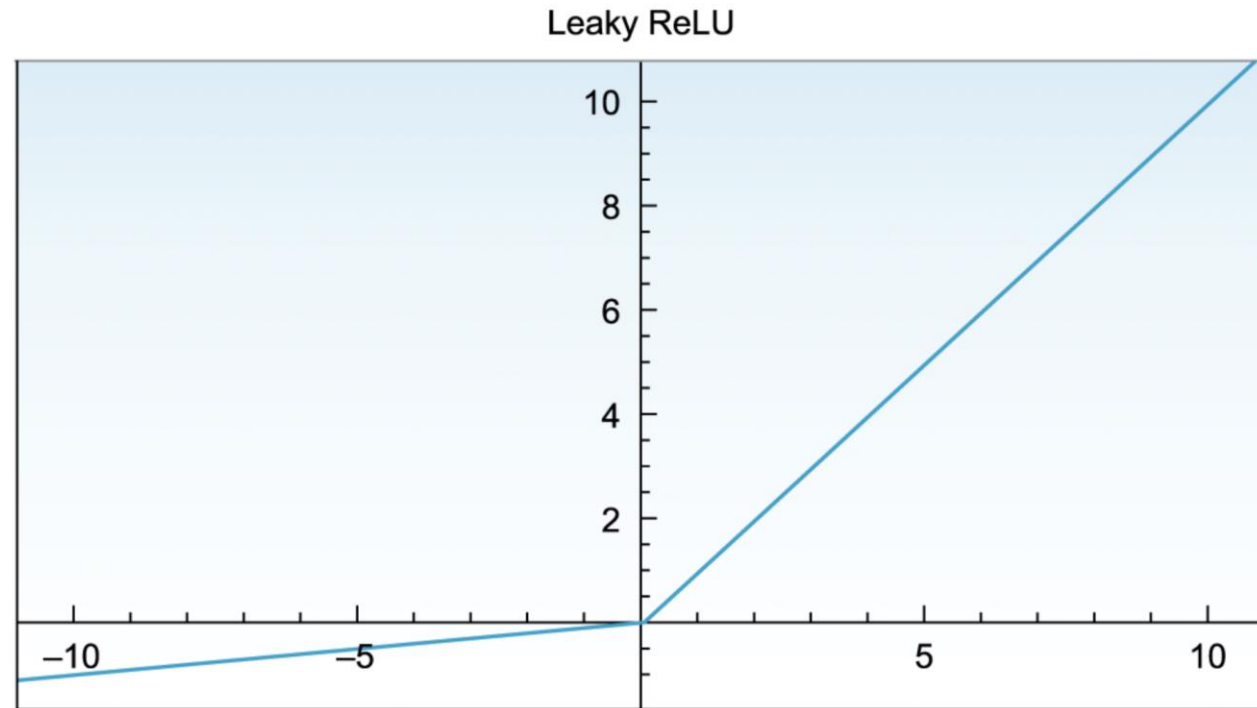
- Activates a node only when the input is greater than zero
- $f(x) = \max(0, x)$
- Trains better as compared to sigmoid and tanh
- Disadvantage
 - Derivative is equal to zero when input x is zero



$$\text{ReLU}(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

Leaky ReLU

- Introduces a small negative slope when $x < 0$
- $f(x) = \max(0.01x, x)$



$$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

Table of Activation functions

- Check in the book, chapter 2

Feedforward process

$$a_1^{(1)} = \sigma(w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3)$$

$$a_2^{(1)} = \sigma(w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{23}^{(1)}x_3)$$

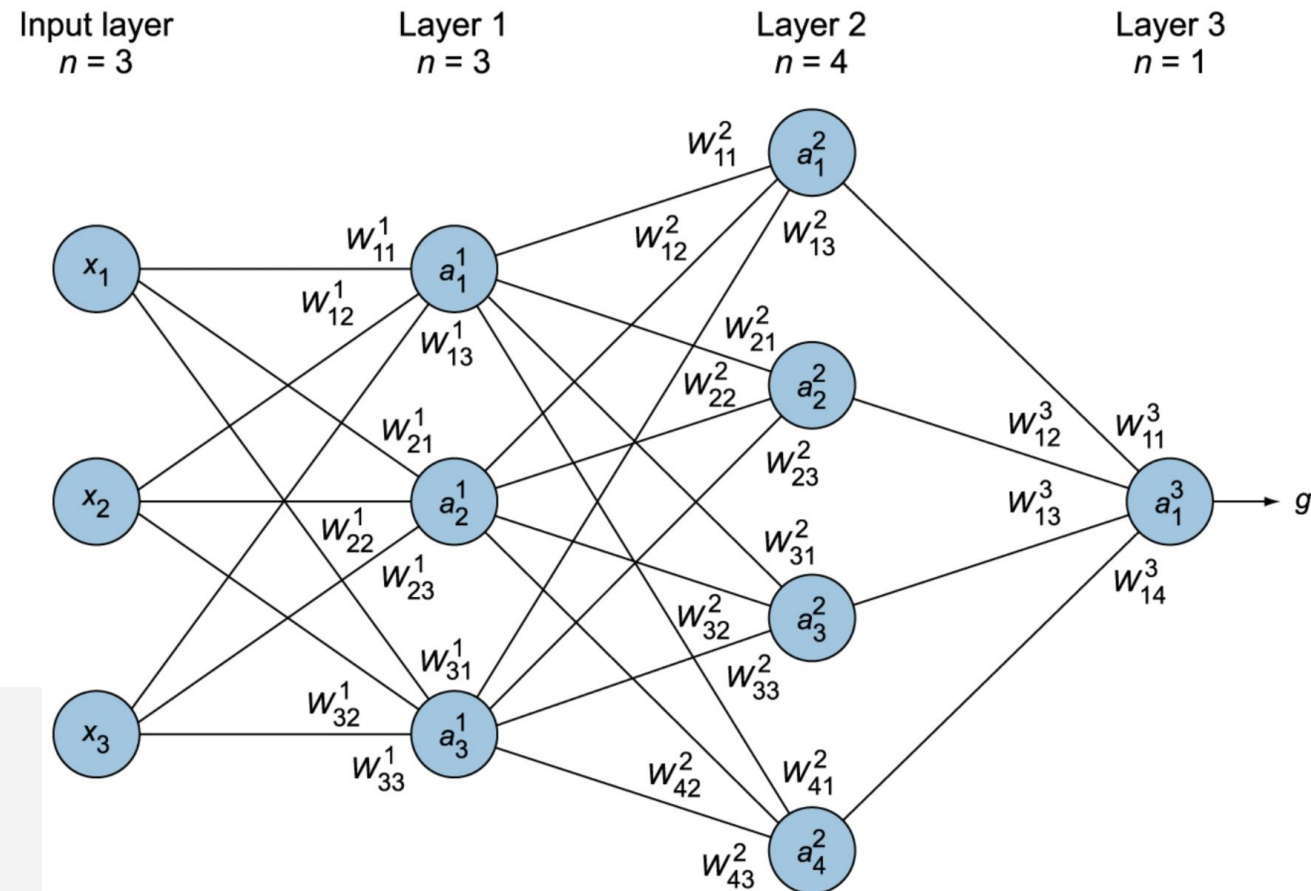
$$a_3^{(1)} = \sigma(w_{31}^{(1)}x_1 + w_{32}^{(1)}x_2 + w_{33}^{(1)}x_3)$$

Then we do the same calculations for layer 2

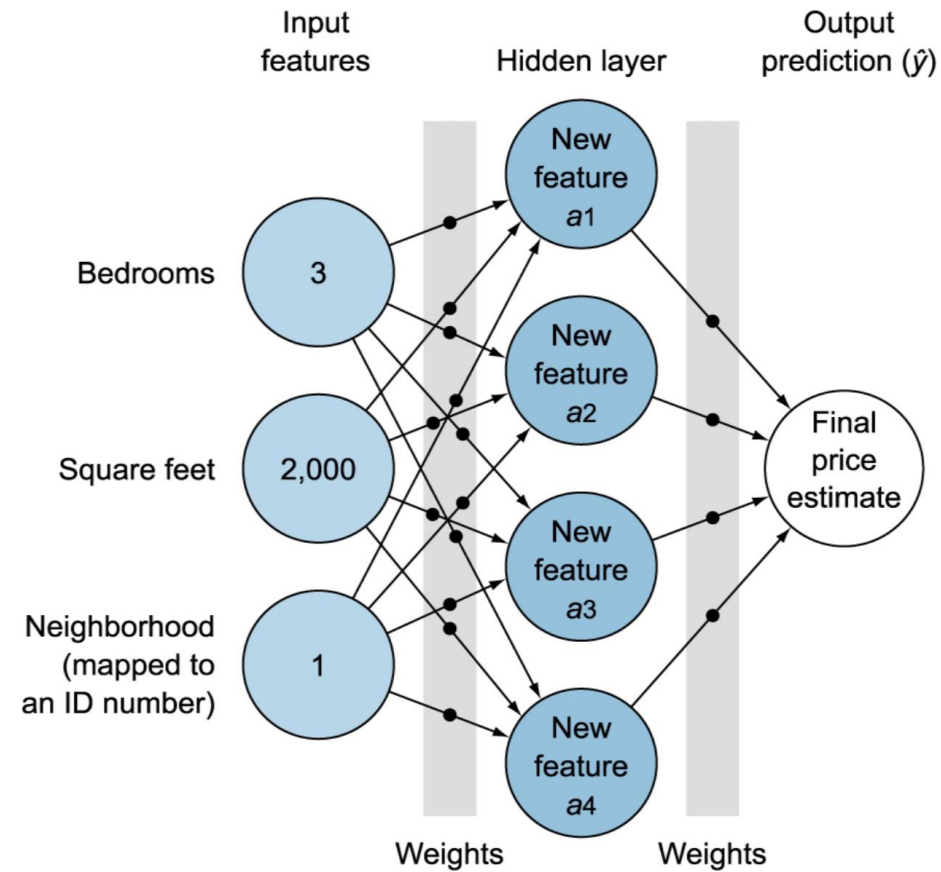
$$(a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, \text{ and } a_4^{(2)})$$

all the way to the output prediction in layer 3:

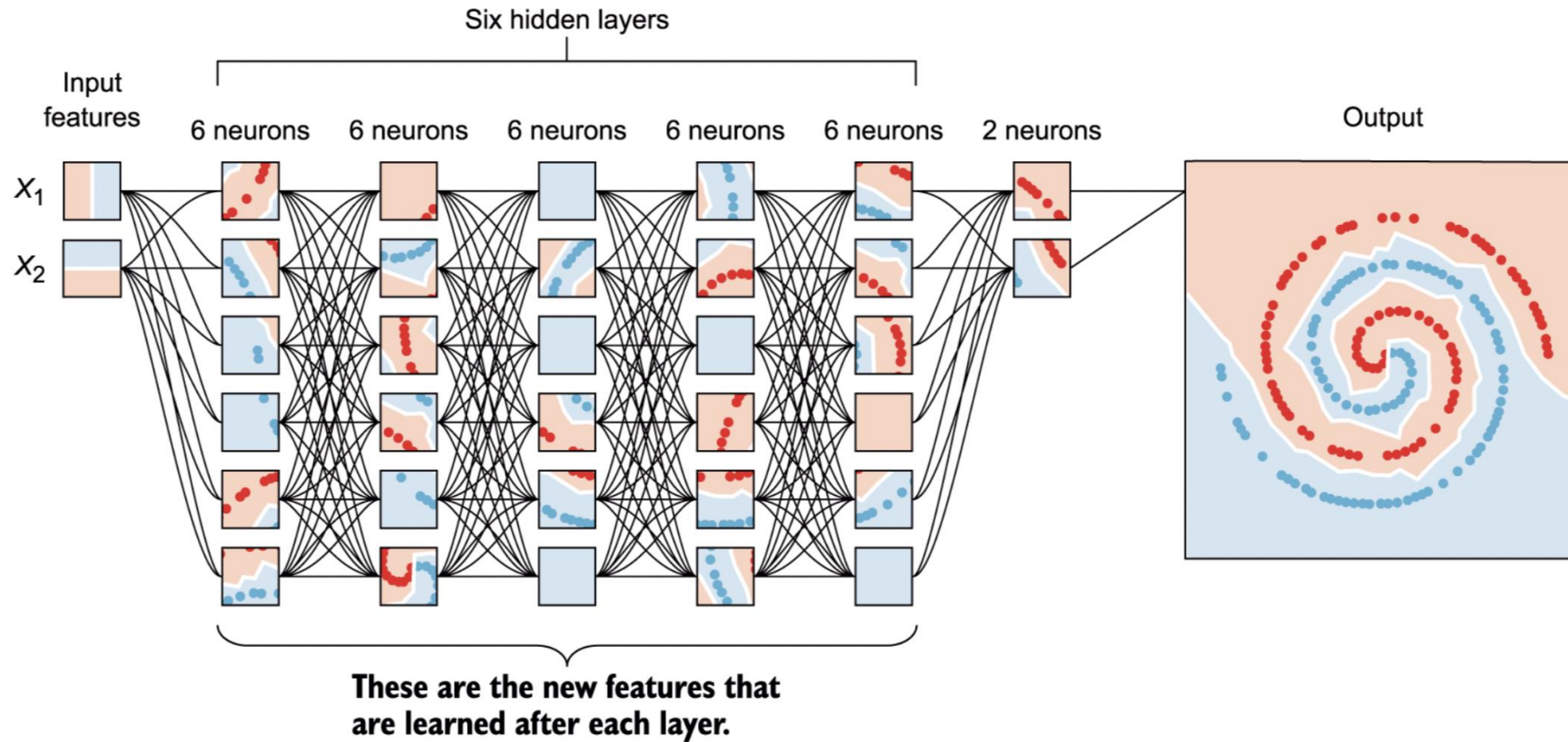
$$\hat{y} = a_1^{(3)} = \sigma(w_{11}^{(3)}a_1^{(2)} + w_{12}^{(3)}a_2^{(2)} + w_{13}^{(3)}a_3^{(2)} + w_{14}^{(3)}a_4^{(2)})$$



Feature learning



Multilayer Perceptron Architecture



Input Layer

Weighted edges

Hidden Layers

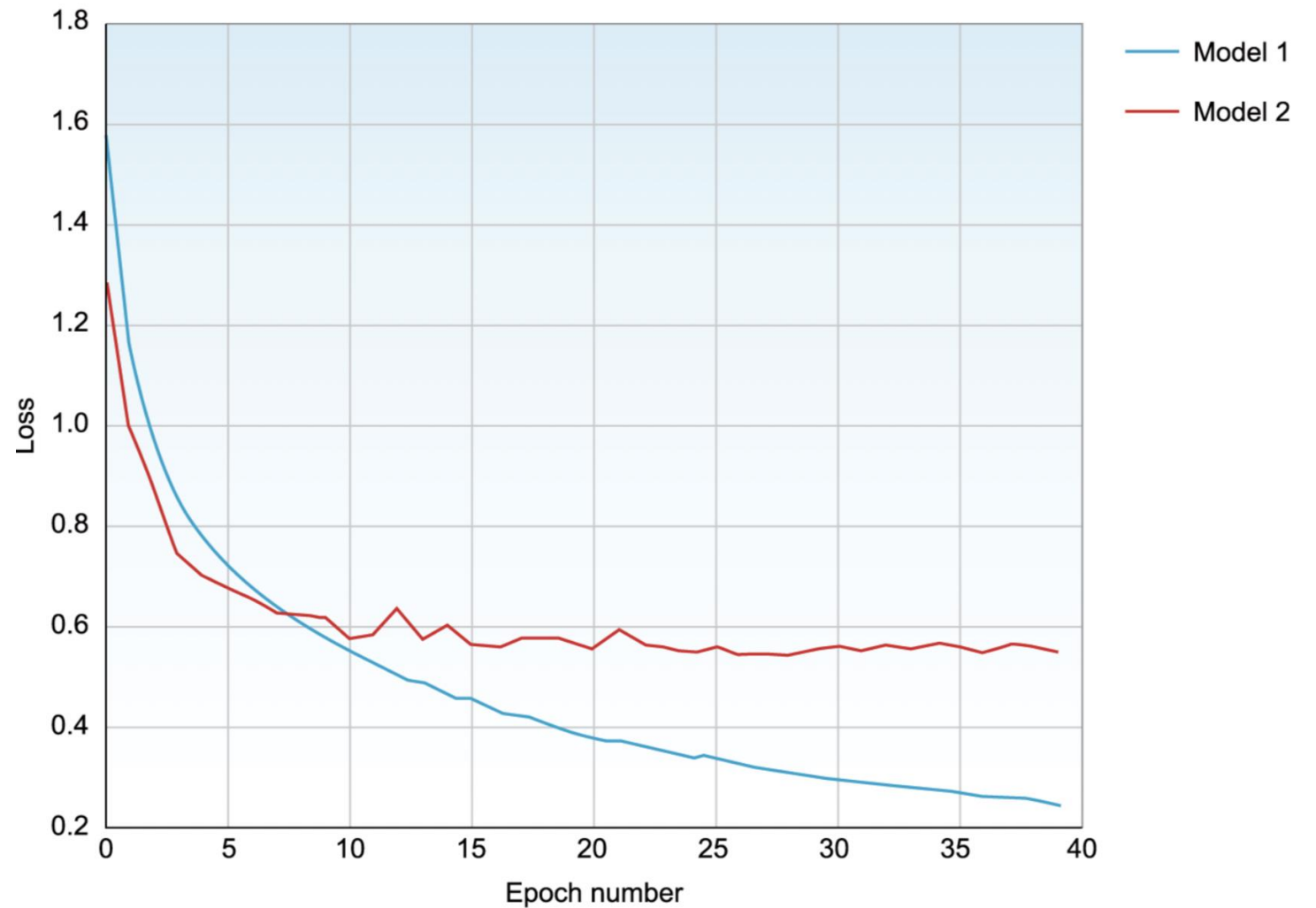
Output Layer

Error/Cost/loss functions

- Measure the error
- Choose the error function
- Deviation of the predicated output from the expected output
 - Large error – low accuracy
 - Small error – better accuracy, model is predicting well
- Importance of defining the error function
 - Helps with calculating the error
 - Goal: Optimize the error function parameters (weights for NN) to reduce the error

Error is always positive

- The error needs to be positive to avoid getting it cancelled while calculating the average error
- Visualization of loss functions for 2 models
- Different loss functions can lead to different error values for the same prediction
 - Mean squared error – used for regression problems
 - Cross entropy – used for classification problems



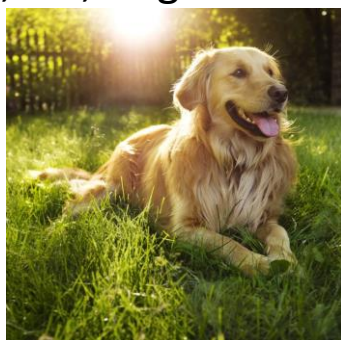
Mean Square Error (MSE)

- Used for regression problems when the output requires to be a real value (e.g. housing price)
- MSE equation
 - $E(W, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$
- Ensures the positive error
- Larger errors are penalized more than smaller errors
- Sensitive to outliers
 - Good for some applications, e.g. stock price
 - Not suitable for some applications e.g. house pricing in the city
 - Use MAE (Mean Absolute Error)
 - $E(W, b) = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$

Notation	Meaning
$E(W, b)$	The loss function. Is also annotated as $J(W, b)$ in other literature.
W	Weights matrix. In some literature, the weights are denoted by the theta sign (θ).
b	Biases vector.
N	Number of training examples.
$\hat{y}_i$	Prediction output. Also notated as hw , $b(x)$ in some DL literature.
y_i	The correct output (the label).
$(\hat{y}_i - y_i)$	Usually called the residual.

Cross Entropy

- Used in classification problems
 - Quantifies the difference between 2 probabilities distributions
- Cross entropy equation,
 - $E(W, b) = -\sum_{i=1}^m y_i \log(p_i)$
 - $y \rightarrow$ target probability
 - $p \rightarrow$ predicated probability
 - $m \rightarrow$ number of classes
- E.g. 3 classes, cat, dog and fish
 - Input



True output	Probability(cat)	P(dog)	P(fish)
	0.0	1.0	0.0

Predicated output - 1

Probability(cat)	P(dog)	P(fish)
0.2	0.3	0.5

Cross Entropy for the predicated output

$$E = - (0.0 * \log(0.2) + 1.0 * \log(0.3) + 0.0 * \log(0.5)) = 1.2$$

Predicated output - 2

Probability(cat)	P(dog)	P(fish)
0.3	0.5	0.2

Cross Entropy for the predicated output

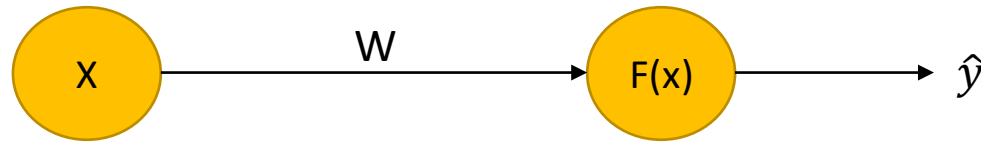
$$E = - (0.0 * \log(0.3) + 1.0 * \log(0.5) + 0.0 * \log(0.2)) = 0.69$$

Cross Entropy for n training samples

$$E(W, b) = - \sum_{i=1}^n \sum_{j=1}^m y_i \log(p_{ij})$$

Errors and Weights

- Perceptron with a single input



- Suppose input $x = 0.3$, true output, $y = 0.8$, predicated output for the given perceptron can be calculated using feedforward process:
 - $y_i = w.x = w.0.3$
 - Corresponding error

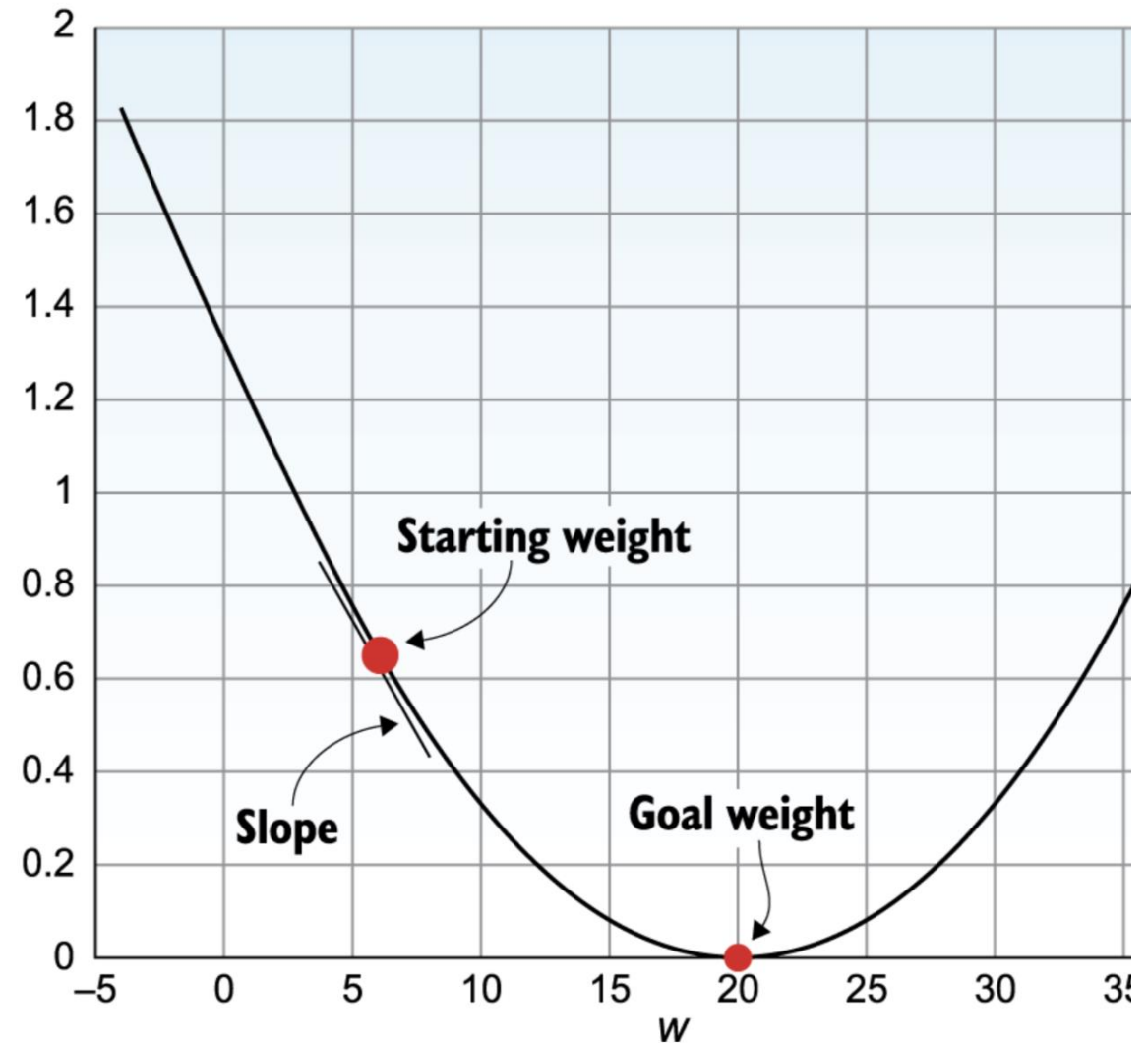
$$\begin{aligned} error &= |\hat{y} - y| \\ &= |(w.x) - y| \\ &= |w.0.3 - 0.8| \end{aligned}$$

Input x and expected output y
and constants

Network learns by adjusting the
weights

Cost/Error function vs weight plot

- Initialize the network with random weights
- Aim to descend the curve and find the optimal value of w to get the minimum error
- The process of iterating through different weight values to reach the minimum error is called as *optimization algorithm*



Reference

- Book – Deep Learning for Vision Systems by Mohamad Elgendy, Manning Publication, November 2020
- Fukushima, Kunihiro. "Cognitron: A self-organizing multilayered neural network." *Biological cybernetics* 20, no. 3 (1975): 121-136.
<https://link.springer.com/article/10.1007/BF00342633>
- Nair, Vinod, and Geoffrey E. Hinton. "Rectified linear units improve restricted boltzmann machines." In *icml*. 2010.